

Stock Market Management System

Technical Documentation

Version: 1.0

Date: 24 May 2026

1. Overview

The Stock Market Management System is a simple MVP application that helps users register, view stock prices, execute buy and sell trades, manage a portfolio, and review transaction history. It provides a clean separation between the frontend and backend while keeping the data model and API minimal for easy understanding and extension.

Key capabilities:

- User registration and login
- View stock prices
- Buy and sell shares
- Portfolio summary and holdings
- Transaction history
- Market analysis snapshot

2. Technology Stack

Frontend

- React 18
- Vite 5
- JavaScript (ES modules)
- CSS (black and blue theme)

Backend

- Node.js
- Express.js
- SQLite (file-based database)
- cors

Authentication

- Simple access token (Bearer token)
- Token stored in localStorage

3. Architecture

3.1 Components

- Frontend Layer: Captures user input and renders dashboard, portfolio, transactions, and market analysis.
- Express Backend Layer: Validates requests and orchestrates portfolio and transaction operations.
- Data Layer (SQLite): Stores users, stock prices, holdings, and transactions.
- MVP Pattern: Models handle data queries, presenters handle business rules, controllers handle requests and responses.

3.2 System Architecture

Frontend Layer -> Express Backend Layer -> SQLite Database UI -> API -> Data

3.3 Request Lifecycle

1. User logs in and receives an access token.
2. Frontend sends requests with Authorization: Bearer .
3. Backend validates the token and user.
4. Backend executes model operations and presenter logic.

5. Backend returns JSON responses.
6. Frontend renders updated views.

4. Data Model

Tables:

- users: name, email, password, token, created_at
- stocks: symbol, name, price, change_percent
- portfolio_holdings: user_id, stock_id, quantity, avg_price
- transactions: user_id, stock_id, type, quantity, price, created_at

Note: Stock prices are seeded and static for the MVP. Trades and holdings are stored in SQLite and persist between runs.

5. API Endpoints

Base URL: <http://localhost:4000>

5.1 GET /api/health

Purpose:

- Service health check. **Response 200:**

```
{  
  "status": "ok"  
}
```

5.2 POST /api/auth/register

Purpose:

- Register a new user and return an access token. **Request fields:**
- name: string (required)

- email: string (required)
- password: string (required) **Sample request:**

```
{
  "name": "Asha",
  "email": "asha@example.com",
  "password": "pass123"
}
```

Sample response:

```
{
  "user": { "id": 1, "name": "Asha", "email":
"asha@example.com" },
  "token": "<access_token>"
}
```

5.3 POST /api/auth/login

Purpose:

- Log in and return a new access token. **Request fields:**
- email: string (required)
- password: string (required) **Sample response:**

```
{
  "user": { "id": 1, "name": "Asha", "email":
"asha@example.com" },
```

```
"token": "<access_token>"
}
```

5.4 GET /api/stocks

Purpose:

- List available stocks with prices and change percent. **Response 200:**

```
[
  {
    "id": 1,
    "symbol": "AAPL",
    "name": "Apple Inc.",
    "price": 186.23,
    "change_percent": 1.2
  }
]
```

5.5 GET /api/market/analysis

Purpose:

- Market snapshot including top movers and losers. **Response 200:**

```
{
  "summary": {
    "marketStatus": "Bullish",
    "avgChangePercent": 0.68,
  }
}
```

```
    "topMover": { "symbol": "NVDA",
"change_percent": 3.1 }
  },
  "topMovers": [ ... ],
  "topLosers": [ ... ]
}
```

5.6 GET /api/portfolio

Purpose:

- Get the current user portfolio summary. **Auth:**
- Requires Bearer token. **Response 200:**

```
{
  "totals": {
    "totalValue": 12450.75,
    "dailyChange": 85.12,
    "dailyChangePercent": 0.68
  },
  "holdings": [ ... ]
}
```

5.7 POST /api/portfolio/buy

Purpose:

- Buy shares and update holdings. **Auth:**
- Requires Bearer token. **Request fields:**
- stockId: integer
- quantity: integer **Response 200:**

- Returns updated portfolio object.

5.8 POST /api/portfolio/sell

Purpose:

- Sell shares and update holdings. **Auth:**
- Requires Bearer token. **Request fields:**
- stockId: integer
- quantity: integer **Response 200:**
- Returns updated portfolio object.

5.9 GET /api/transactions

Purpose:

- List recent transactions. **Auth:**
- Requires Bearer token. **Response 200:**

```
[
  {
    "id": 10,
    "type": "BUY",
    "quantity": 4,
    "price": 186.23,
    "symbol": "AAPL",
    "created_at": "2026-05-24 10:11:12"
  }
]
```

6. Reliability, Security, and Governance

6.1 Reliability

- SQLite data persists locally.
- Consistent JSON response format.
- Health endpoint for monitoring.

6.2 Security and Privacy

- Access tokens are required for portfolio and transaction routes.
- CORS enabled for local development.
- Passwords are stored as plain text in this MVP and must be replaced with hashing for production.

6.3 Recommended Production Controls

- HTTPS and secure transport
- Password hashing (bcrypt)
- Rate limiting
- Audit logging
- Role-based access control

7. Extensibility

- Plug in live market data feeds.
- Add price update scheduler.
- Add alerting and watchlists.
- Expand analytics and reporting.
- Add admin dashboards for monitoring.

8. Conclusion

This MVP provides a clean, modular stock management system with a simple API and a focused UI, making it easy to extend into a production-ready trading or portfolio platform.